



0:12 hello everyone welcomes to master2 teach step by step guide 0:21 in this video we are going to learn what is sequence diagram what are the 0:25 different parts of sequence diagram we will go through one by one how to draw a 0:30 sequence diagram we'll see step by step we will also see 0:34 an example drawing sequence diagram with case study I will try my best to explain 0:39 step by step if you haven't subscribed to our channel please click on the 0:43 subscribe button and subscribe to our YouTube channel ok let's start What is Sequence Diagram 0:47 what is sequence diagram before restarting I would like to make clear 0:51 that the sequence diagram is also called as an even diagram or even scenario the 0:56 UML diagram which shows an interaction between object arranged in the time 1:00 sequence is a sequence diagram sequence diagram listed how the different part of 1:05 system interact with each other to carry out a function and the order in which 1:09 the interaction occur when a particular use case is executed as I have already 1:14 mention in my previous video tutorial of communication diagram sequence diagram 1:19 and the communication diagram are the type of interaction diagram where it 1:22 shows how objects of classes communicate by sending and receiving 1:25 messages also I have already mentioned that these two diagrams communication 1:30 and sequence diagram are isomorphic which means you can convert one diagram 1:34 into another diagram so far we have gone through how to draw use case diagram and 1:39 how do we write high-level and expanded use case description in the previous 1:44 video tutorial if you haven't gone through it please find the link above or 1:48 in the description of this video and go through it before you start sequence 1:52 diagram after the expanded use case description of the major use cases we 1:56 can start either the communication diagram or sequence diagram so that you 2:00 should know what is expanded each case description and how to write expanded 2:05 each case description is a two-way interaction steps written between the 2:08 actor and the system how that particular use case will execute we have already 2:13 drawn the communication diagram in the previous video now let's draw the 2:16 sequence diagram the sequence diagram is an interaction diagram that shows 2:20 details on how operations are carried out like what masses are sent and when it 2:26 can be drawn at different levels of details and to meet different proposes 2:30 at several stages in the development lifecycle 2:34 typically used to represent the detailed object interaction that occurs for one 2:38 use case or for one operation we all know that objects of the classes 2:44 communicate by sending and receiving messages let me show you an

example how 2:48 objects of the classes communicate by sending and receiving messages in coding 2:52 here is class A with method name as method A which will perform certain tasks 2:57 another Class B with method name as method B which will also perform certain 3:02 tasks now another Class C which is the main 3:06 class or controller class here new object a of class A is created also new 3:12 object b of class B is created a dot method A where C says to A that method A 3:19 should be executed answer is equal to b dot method B C says to B that 3:24 method B should be executed and B returns the results to C which is 3:28 store in answer variable in this way object of the class is communicated by 3:32 sending and receiving messages I hope you are more clear about how do the 3:36 object communicate and showing this interaction of the objects message 3:40 sending and receiving stepwise arrange in time sequence is a sequence 3:44 diagram a sequence diagram is structured in such of way that it represents 3:49 a time line that begins at the top and descends gradually to mark the sequence 3:54 of interaction each object has a column and the messages exchanged between them 3:59 are represented with arrows let's go through each and every parts of sequence 4:05 diagram step by step number one dimension a sequence diagram is drawn in Object Dimension 4:11 two dimension horizontal and vertical object dimension the horizontal axis 4:16 shows the elements or objects that are involved in the interaction are listed 4:22 from left to right according to when they take part in the message sequence 4:26 they can be appear in any order in the horizontal axis time dimension the Time Dimension 4:32 vertical axis in the sequence diagram shows the time from top to down note 4:38 that our sequence diagram is all about the ordering not duration so that while 4:43 drawing sequence diagrams we have to careful on timing and ordering but we 4:48 don't mention the duration of time on it actor Actor 4:53 anyone who used to perform a certain function in the system is actor actor 4:58 maybe someone or something that can be a person an organisation another system 5:02 machine or any external device so who or what is going to use our system is 5:08 actors like customer staff admin bank student teacher etcetera the actor is 5:14 representing by the human stick image as same as in use case and communication 5:19 diagram lifeline a lifeline represents an individual participant in the Lifeline 5:25 interaction a lifeline is shown using a symbol that consists of a rectangle 5:30 forming its head followed by a particle line which may be dashes that represents 5:36 a lifetime of the participant they represents that different objects or 5:41 parts that interact with each other in the system during the sequence the 5:46 standard in UML for naming a lifeline follows the following format instance 5:52 name : class name it represents the passes of time as it extends downward as 5:58 the object is destroyed the lifeline ends there no two lifeline notations should 6:04 be overlap each other a lifeline notation with an actor element symbol is

6:09 used when the particular sequence diagram is owned by the use case
 6:14 activations a thin rectangle box on a lifeline represents the time needed
 for 6:20 an object to complete a task simply we can define as the execution of
 an 6:26 operation the top and the bottom of the rectangles are aligned with
 the 6:31 initiations and the completion times respectively 6:34 it reflects the
 time of execution so that the activation box become longer as 6:40 the task is
 longer message a message defines a particular communication Messages
 6:47 between life lines of an interaction which means showing how the
 objects 6:51 interact with each other by sending and receiving messages the
 messages appears 6:56 in sequential order in the lifeline we represent
 messages using arrows we can 7:01 further classify the messages as follows
 synchronous messages synchronous Synchronous Messages 7:06 messages
 represented by a solid line with a solid 7:09 Arrowhead this symbol is used
 when a sender must wait for a response to a 7:14 message before it continues
 the sender wait until the receiver had completed 7:20 the processing of the
 messages the caller continues only when it knows that the 7:25 receiver has
 processed the previous messages i.e it receives a reply 7:31 messages the
 diagram should show the both the call and reply to messages Asynchronous
 Messages 7:37 asynchronous messages an asynchronous message doesn't
 wait for the response 7:42 from the receiver before the sender continue the
 interaction moves forward 7:47 irrespective of the receiver processing the
 previous message or not we use a 7:53 line arrow to represent an
 asynchronous message and only the call should be 7:58 included in the
 diagram return message a return message is used to indicate Return
 Messages 8:02 that the message receiver is done processing the message and
 is returning control over to the message 8:08 caller note that returning
 messages are optional notations and only triggered by 8:14 the synchronous
 messages you can avoid cluttering up your diagram by minimizing 8:19 the
 use of return messages since the return values can be specified in the 8:23
 initial message arrow itself create messes an object can be created at any 8:29
 times of excitation of the process to represent the new object created in the
 8:34 process of execution create a message is used it is represented by a
 dashed 8:39 line with a line Arrowhead and label with word create on it to
 specify 8:44 that it is the crate message symbol reflexive messages sometimes
 an object Reflex 8:49 needs to communicate with the same class methods that
 is when an object sends a 8:54 message to itself it is called the reflexive
 message it is indicated with a 9:00 message Arrowhead that starts and ends
 at the same lifeline as shown in the 9:04 diagram object destruction as soon
 as the object does not need anymore in the Object Destruction 9:10 further
 process object destruction is called which is deleting the object 9:14 object
 destruction represents the request of destroying the lifecycle of 9:18 the
 target lifeline which is shown Cross lines and there will not further 9:24 lifeline

symbols mean dotted line it destroys the occurrence of the object 9:28 in the system comment a comment or a note can be used Comment 9:33 to reflect the various remarks to the elements a comment carries no semantic 9:38 force but may contain information that is useful to the modular the common 9:44 object is our rectangle with a folded over corner as shown in the diagram 9:49 the comment can be linked with a related object with a dashed line now let's move 9:55 further focus of control the focus of control indicates times during Focus of Control 10:01 activation when processing is taking place within that object the focus of 10:06 control is also known as an execution occurrence it represents the period 10:11 during which an element is performing an operation the top and the bottom of the 10:17 rectangle are aligned with the initiations and the completion time 10:21 respectively the focus of control can be shown as follow the white gap in the 10:26 activation reflex the waiting time and shaded with black reflect the execution 10:32 time control object the control object is to manage the collaboration of the Control Object 10:37 objects which gives effect to the use case process the name of the controller 10:41 object is the same name as the use case name like control object for the use 10:46 case enroll student is : enrollstudent don't forget to underline the 10:51 text as well Boundary Object 10:55 boundary object most use cases imply at least one boundary object that managed 11:01 the dialogues between is the actor and the system boundary object will also 11:05 have the same name as the use case name with the additional of UI at last like 11:10 :enroll student UI this object is for the user interface control object and 11:16 boundary object are similar as the communication diagram now let's learn 11:20 further more advanced in the sequence diagram guards when modeling object 11:25 interaction there may be the certain condition that must be met for a 11:29 message to be sent to the object to model conditions like if condition in 11:34 programming we use guard in sequence diagram guards play an important role in 11:38 letting software developer know the constraints attached to the system's or 11:43 a particular process for example in order to withdraw cash from the bank the 11:48 account balance should be greater than zero so balance greater than zero is a 11:53 condition that must be made for this message to be called which can be shown 11:58 in the sequence diagram like this alternative in Guards we represents the if 12:04 condition in the diagram whereas alternative in sequence diagram 12:09 symbolize a choice between two or more messages sequence to model a choice of 12:14 the message with a condition like if-else condition in programming we use 12:19 an alternative the alternative fragment is represented by a large 12:24 rectangle or a frame it is specified by mentioning alt inside the frame name box 12:31 as shown in the diagram to represent alternatives use the label 12:37 rectangle shape with dashes line inside as shown in the figure only the one 12:42 whose condition is true will execute

optional optional is similar to guards 12:49 where the fragments execute only if the supplied condition is true guards are 12:54 used only for a single message whereas optional is used to model fragments of 12:59 the process as shown in the figure it is represented by a large rectangle or a 13:03 frame where opt is labeled inside the box similar to the alternative in the 13:09 alternative there are a if else condition whereas in optional there is 13:12 only if condition same as guards parallel in parallel these fragments is 13:18 run in parallel if the condition is match each fragment or the message will 13:23 execute parallel the notation of parallel is similar to the combination 13:27 of optional and alternatives where there will be the only one condition as 13:31 optional and multiple messages or fragments as alternative it is also 13:37 represented by the large rectangle where Par is label inside the box as shown in 13:42 figure loops the fragments may execute multiple 13:47 times which may be need to model in the diagram sequence diagram has been 13:51 improved with the addition of the loop combinations fragments loops fragments 13:55 is used to represent a repetitive sequence and it is very similar in 14:00 appearance to the optional for loops draw a frame and inside the frame's name 14:06 box label with text loop inside the frame context area the loop guard or 14:11 condition is placed towards the top left corner as shown in figure the loop 14:17 sequence of the messages is placed in the reminder of the frames context area 14:21 in the loops there can be two special condition tested against in addition 14:26 through the standard boolean test the special condition our minimum iterations 14:31 written as min int is equal to the number like min int is equal to 1 and 14:36 the maximum iteration written as max int is equal to number like max int is 14:43 equal to ten with a minimum iteration guard the loop must execute at least the 14:49 number of time indicated whereas the maximum iteration condition the number 14:54 of loop activations cannot exceed the number sequence diagram 15:01 the entire sequence diagram is surrounded by the box or frame and 15:04 inside the frame name label with text SD : process name 15:13 beside this there are other notation like Negative the fragments shows an invalid 15:19 interaction another type is reference reference refers to an interaction 15:25 defined on the another diagram the frame is drawn to cover the lifeline involved 15:30 in the interactions you can define parameters and a return value so using 15:35 this all elements we can draw the sequence diagram how to draw sequence This is the transcript of a video on sequence diagram. summarize it

Here's a **clear and structured summary** of the transcript you shared about *Sequence Diagrams*:

Sequence Diagram – Summary

◆ What is a Sequence Diagram?

- A **UML interaction diagram** that shows how objects communicate through messages arranged in **time sequence**.
- Also called an **event diagram** or **event scenario**.
- Demonstrates **order of interactions** during the execution of a use case or operation.
- Isomorphic to **communication diagrams** (can be converted into one another).

◆ Purpose & Usage

- Represents **detailed object interactions** for a single use case or operation.
- Shows **messages sent/received**, their **order**, and **conditions**.
- Used at different development stages for **design and analysis**.

◆ Key Elements

1. Dimensions

- **Horizontal (Object dimension):** Shows participating objects/actors.
- **Vertical (Time dimension):** Shows time from **top to bottom** (ordering, not duration).

2. Actors

- External entities interacting with the system (e.g., customer, admin, bank).

3. Lifeline

- Represents a participant (object/actor).
- Notation: *instanceName* : *ClassName*.

4. Activation (Execution occurrence)

- Rectangle on a lifeline showing the period an object performs a task.

5. Messages

- **Synchronous:** Solid line + solid arrowhead (sender waits for reply).

- **Asynchronous:** Solid line + open arrowhead (sender doesn't wait).
- **Return message:** Optional dashed arrow back to caller.
- **Create message:** Dashed line with *create*.
- **Reflexive message:** Object calls its own method.

6. Object Destruction

- End of an object's life, shown by a cross (X).

7. Comments/Notes

- Rectangle with folded corner, linked by dashed line.

8. Focus of Control

- Narrow rectangle showing when an object is active (can show waiting vs. execution).

9. Special Objects

- **Control object:** Manages interactions (e.g., `:enrollStudent`).
- **Boundary object:** Manages dialogue with the actor/UI (e.g., `:enrollStudentUI`).

◆ Advanced Features (Fragments)

- **Guards:** Conditions for message execution (e.g., `balance > 0`).
- **Alternative (alt):** If-else choices.
- **Optional (opt):** Executes only if condition true.
- **Parallel (par):** Concurrent execution.
- **Loop (loop):** Repeated execution (with min/max iterations).
- **Negative:** Shows invalid interaction.
- **Reference:** Refers to another diagram's interaction.

◆ Final Notation

- The entire diagram is enclosed in a **frame**, labeled:
`sd : ProcessName.`

✅ **In short:** A sequence diagram visualizes *who talks to whom, when, and in what order*, helping model system behavior step by step.

Do you want me to also **draw a simple example diagram** (e.g., *Login Use Case*) so you get a visual along with this summary?



more detailed explanation with explaining what the thing is and examples

Perfect 🌸 Let's go step by step with **detailed explanations** of each concept in a **Sequence Diagram**, along with **examples** so you can connect the theory with practice.



Sequence Diagram – Detailed Explanation with Examples

♦ 1. What is a Sequence Diagram?

A **Sequence Diagram** is a type of **UML (Unified Modeling Language)** interaction diagram.

It shows **how objects/actors interact with each other** through messages in a **time-ordered manner**.

👉 Think of it as a *timeline of conversations* between system components.

Example:

- In an **online shopping system**, when a customer places an order:
 1. Customer clicks "*Place Order*".
 2. System checks product availability.
 3. System verifies payment.
 4. Confirmation is sent to customer.

The sequence diagram will show *who sends a message to whom* (Customer → System → Payment Gateway → Customer) and in what order.

◆ 2. Dimensions of a Sequence Diagram

a) Object Dimension (Horizontal Axis)

- Shows all the **participants** (actors and objects) from left to right.
- Order doesn't matter, but typically **actor (user) is on the left**.

b) Time Dimension (Vertical Axis)

- Flows **top to bottom**.
- Represents **order of events** (not duration).

👉 Imagine it as a "timeline of actions."

◆ 3. Main Elements

a) Actor

- Anything outside the system that interacts with it.
- Could be a **person, machine, external system, or organization**.

Notation: Stick figure.

Example: In an ATM sequence diagram, the **Customer** is the actor.

b) Lifeline

- Represents an **instance of a class/actor** participating in the interaction.
- Shown as a **rectangle (object) with a vertical dashed line** extending downward.

Format: `objectName : ClassName`

- Example: `cust : Customer`

👉 Lifeline = "The life of an object/actor during the interaction."

c) Activation (Execution Occurrence)

- A **narrow rectangle** drawn on a lifeline.
- Shows the period when an object is **active/performing a task**.
- Longer rectangle = longer task.

Example: When **PaymentGateway** processes a credit card transaction, its lifeline shows a rectangle during that execution.

d) Messages

Messages show communication between lifelines.

Types:

1. Synchronous Message

- Solid line + filled arrowhead.
- Sender waits for a response.
- Example: **Customer** → **ATM** : **enterPIN()** → ATM must respond before customer continues.

2. Asynchronous Message

- Solid line + open arrowhead.
- Sender does not wait.
- Example: **User** → **NotificationService** : **sendEmail()** (user doesn't wait for reply).

3. Return Message

- Dashed line with arrowhead.
- Optional (to avoid clutter).
- Example: **Database** → **System** : **return account details**.

4. Create Message

- Dashed line with «*create*» label.

- Represents creation of a new object.
- Example: `System` → `Session` : `create()`.

5. Reflexive Message

- When an object sends a message to itself.
 - Arrow loops back to the same lifeline.
 - Example: `Calculator` : `computeTotal()`.
-

e) Object Destruction

- Shown with a big "X" at the bottom of a lifeline.
 - Represents termination of the object.
 - Example: Temporary session object being destroyed after logout.
-

f) Comment (Note)

- Used to add **extra explanation**.
 - Shown as a rectangle with a folded corner.
 - Example: A note explaining "Password must be 8+ characters."
-

g) Focus of Control

- A **thin rectangle on a lifeline**.
 - Indicates when an object is actively processing a message.
-

h) Special Objects

- **Control Object**: Manages workflow.
 - Example: `:OrderController` manages the order process.
 - **Boundary Object**: Interfaces with the actor (usually UI).
 - Example: `:LoginUI`.
-

◆ 4. Advanced Concepts (Fragments)

Sometimes processes have conditions, loops, or choices. Sequence diagrams represent these using **fragments (frames)**.

1. Guard Conditions

- Show conditions for execution.
- Example: `[balance > 0] withdraw()`.

2. Alternative (alt)

- If-else conditions.
- Example:
 - If login is valid → show dashboard.
 - Else → show error.

3. Optional (opt)

- Executes only if condition true.
- Example: `[isFirstLogin] showTutorial()`.

4. Parallel (par)

- Processes run simultaneously.
- Example: ATM can **dispense cash** while also **printing receipt**.

5. Loop (loop)

- Repeated interactions.
- Example: `[for each item] addToCart()`.

6. Negative

- Shows invalid interaction.
- Example: **InvalidPIN** scenario.

7. Reference (ref)

- Refers to another sequence diagram.
- Example: "ProcessPayment" fragment referring to a separate diagram.

◆ 5. Complete Diagram Notation

- The entire sequence diagram is wrapped in a **frame**.

- Labeled as: **sd : UseCaseName.**

Example:

sd : Withdraw Cash

Example – ATM Cash Withdrawal

Actors & Objects:

- Customer (actor)
- ATM (boundary object)
- Bank Server (control object)
- Account (entity object)

Flow:

1. Customer → ATM: insertCard()
2. ATM → Customer: requestPIN()
3. Customer → ATM: enterPIN()
4. ATM → BankServer: validatePIN()
5. BankServer → Account: checkBalance()
6. Account → BankServer: return balance
7. BankServer → ATM: approveTransaction()
8. ATM → Customer: dispenseCash()
9. ATM → Customer: printReceipt()
10. ATM → Session: destroy()

This sequence visually shows **step-by-step order** and **interactions**.

 In summary:

A **sequence diagram** is like a *movie script of your system's behavior*.

It tells **who talks to whom**, **what is said**, and **in what order**, with conditions, loops, and parallel activities clearly shown.

Would you like me to **draw a diagram (ATM Login/Withdraw example)** in UML notation so you can *see the flow visually* along with this detailed explanation?



more detailed explanation

Got it 🌿 You want a **deep dive** — not just definitions, but also **why each element exists, how it is used, and examples from real-life systems**. Let's go into a **comprehensive, textbook-like explanation** of **Sequence Diagrams**.

Sequence Diagram – Comprehensive Explanation with Real Examples

♦ 1. Introduction to Sequence Diagram

A **Sequence Diagram** is part of the **UML (Unified Modeling Language)** family. It belongs to the group of **interaction diagrams** (alongside *Communication Diagrams*).

👉 It answers the question:

“How do system components interact step by step over time to fulfill a use case?”

It focuses on:

- **Order of events** (time-based interactions).
- **Flow of messages** between objects/actors.
- **Control of logic** (conditions, loops, parallel flows).

💡 Think of it as a **storyboard of a software process**: like a script where characters (objects/actors) exchange dialogues (messages) in a specific order.

♦ 2. Why Do We Use Sequence Diagrams?

- To **visualize object interactions** clearly.
- To **validate system behavior** with stakeholders.
- To help developers **understand and implement logic**.

- To **find missing requirements** by simulating use cases.
- To **document existing systems** for analysis or teaching.

Real Example:

In an **E-commerce System**, before implementing “Place Order” feature:

- A sequence diagram ensures everyone understands that:
 - Customer → Website → Payment Gateway → Warehouse → Customer
 - Without this, developers may miss steps like “update stock” or “send email.”
-

◆ 3. Dimensions of a Sequence Diagram

a) Object Dimension (Horizontal Axis)

- Shows **all the participants** (actors, boundary objects, controllers, entities).
- Arranged left → right.
- Example:
`Customer | UI : ShoppingPage | Controller : OrderManager | Entity : Inventory
| Entity : PaymentGateway`

b) Time Dimension (Vertical Axis)

- Shows **progress of time** from top → bottom.
 - NOT about *duration*, but *order*.
 - Example:
 1. Customer requests product.
 2. System checks stock.
 3. System confirms.
-

◆ 4. Core Elements and Their Details

Actor

- External participant interacting with the system.

- Can be a **person, machine, or external system**.
- Shown as a **stick figure**.

💡 Example:

- In a **Bank ATM system**: **Customer** (actor).
- In a **Library system**: **Librarian** and **Student**.

2 Lifeline

- Represents the **existence of an object or actor** in the interaction.
- Format: **objectName : ClassName**
- Shown as a **rectangle with a dashed vertical line** beneath it.
- Lifeline ends when the object is destroyed (shown with X).

💡 Example:

- **cust : Customer**
- **pg : PaymentGateway**

3 Activation (Execution Occurrence)

- Narrow rectangle on lifeline.
- Represents **active execution** of an operation.
- The height shows how long the execution lasts.

💡 Example:

When ATM verifies PIN, the **BankServer's lifeline** will have a long activation box representing the time taken to validate credentials.

4 Messages

Arrows that show communication. Different types:

a) Synchronous Message

- Solid line + filled arrowhead.

- Sender **waits** until receiver finishes.

💡 Example:

- **ATM → BankServer : validatePIN()**
(Customer can't proceed until bank server responds).

b) Asynchronous Message

- Solid line + open arrowhead.
- Sender **does not wait** for a response.

💡 Example:

- **User → NotificationService : sendEmail()**
(User doesn't wait for email to be sent).

c) Return Message

- Dashed line with arrowhead.
- Shows a response.
- Often omitted to reduce clutter.

💡 Example:

- **BankServer → ATM : validPIN.**

d) Create Message

- Dashed line with label *create*.
- Indicates creation of a new object.

💡 Example:

- **System → Session : create().**

e) Reflexive Message

- An object sends message to itself.
- Arrow loops back to same lifeline.

💡 Example:

- `OrderController : calculateTotal()`.

5 Object Destruction

- Lifeline ends with a large "X".
- Indicates termination of object.

💡 Example:

- After logout, `Session` object is destroyed.

6 Comments/Notes

- Rectangle with folded corner.
- Used to clarify details.

💡 Example:

Note: *"PIN must be 4 digits."*

7 Focus of Control

- Another term for activation box.
- Can show active (shaded) and waiting (empty).

8 Special Objects

- **Boundary Object:** Interfaces between system & actor.
 - Example: `:LoginUI` (accepts input from user).
- **Control Object:** Manages workflow.
 - Example: `:PaymentController`.
- **Entity Object:** Represents data storage.
 - Example: `:Account`.

👉 Together, these form the **Model-View-Controller (MVC)** pattern in diagrams.

◆ 5. Advanced Constructs (Combined Fragments)

Sometimes interactions depend on conditions, choices, loops, or parallelism. UML provides **fragments** for these.

a) Guard Condition

- Condition placed before message.
- Format: `[condition] message`.

💡 Example:

`[balance > 0] withdrawCash()`.

b) Alternative (alt)

- Like *if-else*.
- Only one path executes.

💡 Example:

```
csharp

alt
[PIN correct] → proceed with transaction
[PIN wrong] → show error
```

c) Optional (opt)

- Like *if only*.
- Executes only if condition true.

💡 Example:

`[isFirstLogin] showTutorial()`.

d) Parallel (par)

- Multiple processes run simultaneously.

💡 Example:

ATM can **dispense cash** and **print receipt** at same time.

e) Loop (loop)

- Repetition of messages.

💡 Example:

[for each item in cart] addToCart().

Special conditions:

- `minint = 1` → must run at least once.
 - `maxint = 5` → cannot run more than 5 times.
-

f) Negative

- Shows invalid interaction.

💡 Example:

[InvalidPIN] accessDenied().

g) Reference (ref)

- Refers to another diagram.

💡 Example:

`ref` : `ProcessPayment` (instead of repeating all steps).

◆ 6. Putting It All Together – Example

Use Case: ATM Cash Withdrawal

Actors & Objects:

- Customer (actor)
- ATM (boundary)
- BankServer (control)

- Account (entity)

Flow:

1. Customer → ATM : insertCard()
2. ATM → Customer : requestPIN()
3. Customer → ATM : enterPIN()
4. ATM → BankServer : validatePIN()
5. BankServer → Account : checkBalance()
6. [balance > 0] ATM → Customer : dispenseCash()
7. ATM → Customer : printReceipt()
8. ATM → Session : destroy()

This shows **actors, messages, guards, and object destruction** all together.

Final Thoughts

A **Sequence Diagram** is more than just arrows:

- It's a **conversation map** of the system.
 - It blends **actors, objects, time, conditions, and logic**.
 - It's essential for both **requirements analysis** and **system design**.
-

Would you like me to now **create a UML diagram image** (ATM withdrawal or Online Shopping Order) so you can *see everything visually* instead of just text?